UNIVERSITY COLLEGE LONDON

EXAMINATION FOR INTERNAL STUDENTS

MODULE CODE : **COMP0008**

ASSESSMENT Pattern: **COMP0008A5UE**

MODULE NAME : **Computer Architecture and Concurrency**

LEVEL: : **Undergraduate**

DATE: : **09-May-2023**
TIME : **14:30**
DURATION : **03:15**

Late submission is permitted for Controlled Conditioned exams but late penalties will apply - any submissions that are up to 40 minutes late will be penalised, after which no submissions will be accepted under any circumstances.
You must ensure to allow sufficient time to upload and hand in your work

This paper is suitable for candidates who attended classes for this module in the following academic year(s):

Year
2022-23

Duration	<< Exam Duration>>
Additional time for converting handwritten notes to PDF where applicable	**15 mins**
Upload window	**20**
Total time	**3 Hours 35 mins**

Additional material	N/A
Special instructions	You should submit typed answers (i.e., not handwritten) where possible, unless you hold a relevant SoRA. Please, submit your answers as a single PDF file.

TURN OVER

Computer Architecture and Concurrency, COMP0008 (A6U)

Main Summer Assessment 2022/23

Suitable for Cohorts: 2022/23, 2021/22

This assessment includes two parts. Part 1 has 2 sets of multiple choice questions and 1 MIPS programming question. Part 2 includes 2 questions with 5 sub-questions each. Answer ALL the questions, and upload your answers on the UCL exam platform.

Important: You should submit typed answers (i.e., not handwritten) where possible, unless you hold a relevant SoRA. Please, submit your answers as a single PDF file.

Marks: Marks for all questions are indicated in square brackets.
For multiple choice questions, you will be awarded full marks for each correct answer, partial marks for no answer, and zero marks for wrong answers. So, DO NOT GUESS – it is in your interest! Marks for multiple choice questions are indicated by two values: the first value indicates the marks you will be awarded for a correct answer, and the second value corresponds to the marks you will be given in the case of no answer. For example, [(+5,+2) marks] next to a multiple choice question specifies that a correct answer to that question is worth 5 marks, and no answer is worth 2 marks.

Part ONE: Computer Architecture

Theoretical Questions

In your submission, specify your answers using the following template.
You will get zero marks on this part if you don't follow this template!

Part One: Computer Architecture

Q1a: $< your-answer >$
Q1b: $< your-answer >$
Q1c: $< your-answer >$

Q2a: $< your-answer >$
Q2b: $< your-answer >$

Note that your submission must include the sentence "Part One: Computer Architecture" before your answers and a comma after each of your answers. For each question, replace $< your-answer >$ in the above template with A, B, C, or D, depending on which answer you consider correct. Any other text (including no text at all) followed by a comma will be considered as no answer to the corresponding question.

Question 1. In this question, we will scrutinize the jump and branch instructions.

Suppose that either `beq` or `j` instruction (i.e., either a branch-on-equal or jump) is located at address A. We are interested in what the PC value could be after the instruction is executed. For example, $(A+4)$ is possible for both instructions, as both may result in increasing the PC value by 4.

Q1a. Some word-aligned addresses (i.e., those in $\{0,4,8,\ldots,2^{32}-4\}$) are **not** reachable by either jump or branch-on-equal.

[(+5,+2) marks]

A) For any value of A (any address).
B) For some values of A but not all.
C) For no value of A.
D) It depends on whether the previous instruction was `jr`.

Q1b. There are some addresses reachable via jump but not via branch-on-equal.

[(+5,+2) marks]

A) For any value of A (any address).
B) For some values of A but not all.
C) For no value of A.
D) It depends on whether the previous instruction was `add`.

Q1c. There are some addresses reachable via branch-on-equal but not via jump.

[(+5,+2) marks]

A) For any value of A (any address).
B) For some values of A but not all.
C) For no value of A.
D) It depends on whether the previous instruction was `add`.

Question 2. In many applications, it is useful to accurately represent real numbers that are bigger than or equal to 1.

Georgie Bitburger (GB), your 0008 classmate, suggested an alternative representation to the IEEE 754 we saw in class that he claims is more suitable for this case.

In GB's system, each number is represented using a 32-bit bitstring, the same as a single-precision floating point. Let us denote the largest real number representable using an IEEE 754 floating point as R (we saw that $R \approx 3.4 \times 10^{38}$, although the value is irrelevant for this question.)

Denoting $(b = R^{\frac{1}{2^{32}-1}} = R^{\frac{1}{4294967295}})$, GB's system associates a bitstring $x \in \{0, 1, \ldots, 2^{32}-1\}$ with the value b^x. For example, $x = 0$ represents the value $b^0 = 1$, while $x = 2^{32}-1$ represents the value R.

Intuitively, GB suggests that it is better to have the representable values growing exponentially as opposed to having an exponent-mantissa representation.

As a reminder, if a real value $r > 0$ is approximately represented as $\widetilde{r}$, we say that the relative error is $\frac{|r-\widetilde{r}|}{r}$. For example, if π is represented as 3.1, the relative error is approximately $0.04/\pi \approx 0.013$.

Q2a. Suppose that we want to represent the number r using GB's system. Which of the following **is false**?

[(+5,+2) marks]

A) For all $r \in [1, R]$, there exists a representation x that represents a value satisfying $b^x \in [r, r \cdot b]$.
B) For all $r \in [1, R]$, there exists a representation x such that the relative error satisfies $\frac{|b^x - r|}{r} \in [0, \sqrt{b}]$.
C) For all $r \in [1, R]$, there exists a representation x that represents a value satisfying $b^x \in [\frac{r}{\sqrt{b}}, r]$.
D) If $r \leq \sqrt{R}$ then $b^x \leq r$ for all representations $x \leq 2^{30}$.

Q2b. Which of the following is true about the pros and cons of GB's representation? Compared to IEEE 754 floating point:

[(+5,+2) marks]

A) GB's system has lower relative errors for small real numbers but higher errors on large ones.
B) GB's system has higher relative error for all values but is able to perform multiplication operations without losing precision.
C) GB's system has lower error for large real numbers, but addition operations and rounding down to the closest integer are more complex as they cannot be implemented using simple CPU operations such as integer addition and bitwise instructions.
D) GB's system is more efficient for arithmetic operations (because it has just one field x as opposed to exponent and mantissa) but has a higher precision loss for additions.

MIPS Programming

In this section, you are asked to write MIPS code. You are allowed (and encouraged!) to use MARS to check your code.
In your submission, provide your answers using the following template. Notice that you are required to start Q3 on a new page.
You will get zero marks on this part if you don't follow this template!

$< new - page >$
Q3:
$< your - code >$
$< new - page >$

Here, $< your - code >$ must start on a line separate from the question number and must work on the MARS simulator if copy-pasted (*without any manual formatting afterward*). Additionally, your code must not span across page boundaries. That is, you should start your code (preceded with the question number as above) on a new page, but it must not proceed to any additional pages. If needed, you may use smaller font size to fit more lines of code into a page. For example, try copy-pasting the following code into your MARS:

```
.data
msg: .asciiz "\nGood code!\n"
.text
li $v0, 4 # print string
la $a0, msg #pointer to the message
syscall
```

If you choose to answer with 'I don't know', you will be awarded with 5 marks (but none will be given for a wrong solution.)
Academic honesty: All code you submit must be written by you alone. Copying of code from student to student is a serious infraction; the course staff use extremely accurate plagiarism detection software to compare code submitted by all students and identify instances of copying of code; this software sees through attempted obfuscations such as renaming registers and reformatting, and compares the actual parse trees of the code. Rest assured that it is far more work to modify someone else's code to evade the plagiarism detector than to write code yourself!

Question 3. The *number of trailing zeros* of a bitstring is the number of consecutive least-significant zero bits. For example, $0xBA000000 = 0b10111010000000000000000000000000$ has 25 trailing zeros while $0x0C00FEE0 = 0b11000000000011111111011100000$ has 5.

In this question, your submitted code must start with:

```
li $t0, 0xBABA0000
```

Your task is to *efficiently* calculate the number of trailing zeros in the register $t0. Note that your code may be checked on different $t0 inputs and thus should perform the calculation and not print the result for the above input. The output should be just the number of trailing zeros, e.g., '17'.

Your solution will be marked on outputting the correct value and on efficiency. We will use the MARS instruction counter to determine the number of instructions *executed* (and *not* lines of code), and marks will be deducted for inefficient solutions.

[(+25,+5) marks]

Part TWO: Concurrency

This section contains 2 questions, each with 5 sub-questions. In your submission, specify your answers according to the template below. Additional instructions on the format of your answers are provided in each question.
You will lose marks if you don't follow the specified templates!
For each question, replace $< your-answer >$ in the following template with your actual answer to the question. **Be brief** in your explanations: write one or two sentences per question.

Part Two: Concurrency

Q4a Answer:
$< your-answer >$

Q4b Answer: $< your-answer >$
Q4b Explanation: $< your-answer >$

Q4c Answer: $< your-answer >$
Q4c Explanation: $< your-answer >$

Q4d Answer: $< your-answer >$
Q4d Explanation: $< your-answer >$

Q4e Answer: $< your-answer >$
Q4e Explanation: $< your-answer >$

Q5a Answer:
$< your-answer >$

Q5b Answer: $< your-answer >$
Q5b Explanation: $< your-answer >$

Q5c Answer: $< your-answer >$
Q5c Explanation: $< your-answer >$

Q5d Answer: $< your-answer >$
Q5d Explanation: $< your-answer >$

Q5e Answer: $< your-answer >$
Q5e Explanation: $< your-answer >$

Question 4. Consider the following code.

```
public class Student {
  public String id;
  private String name;
  private volatile boolean active;
  private ArrayList<Exam> exams;

  public Student(String id, String name) {
    this.id = id;
    this.name = name;
    this.active = false;
    this.exams = new ArrayList<Exam>();
  }

  public synchronized String getName() {
    return this.name;
  }

  public synchronized void register(Module module) {
    module.addStudent(this.id);
    this.active = true;
  }

  public synchronized void addExam(Exam exam) {
    this.exams.add(exam);
  }

  public synchronized ArrayList<Exam> getTranscript() {
    return new ArrayList<Exam>(this.exams);
  }

  public void printTranscript() {
    if (! this.active){
      System.out.println("(currently inactive)");
    }
    synchronized(this){
      System.out.println("Student " + this.name);
      for (Exam e: this.exams){
        System.out.println(e.toString());
      }
    }
  }
}
```

```
public class Exam {
  private String examId;
  private int mark;

  public Exam(String id, int mark){
    this.examId = id;
    this.mark = mark;
  }

  public String toString() {
    return "Exam " + this.examId + ": " + this.mark;
  }

  public synchronized void update(String newId, int newMark) {
    this.examId = newId;
    this.mark = newMark;
  }
}

class Module {
  private ArrayList<String> students;

  public Module(){
    students = new ArrayList<String>();
  }

  public synchronized void addStudent (String studentId){
    students.add(studentId);
  }

  public void registerStudent (Student newStud){
    newStud.register(this);
  }
}
```

a. Discuss the visibility properties of every instance field defined in `Student` class, following the below format:

> Q4a Answer
> Instance field: *<field name>*
> Method or methods where the field is published: *<name of the methods>*
> Is the field safely published? *<yes or no>*
> Possible visibility problems for this field: *<one or two sentences>*
> *<empty line>*

[8 marks]

b. Focus on the output of the method `printTranscript()`. Can the instruction `System.out.println("(currently inactive)")` ever be executed on any `Student` instance S after `module.addStudent(this.id)` is executed on the same instance S?

If the above is possible, describe an interleaving leading to such output. Otherwise, briefly motivate why the above sequence of actions is not possible.

Report your answer in the following format.

Q4b Answer: *<yes or no>*
Q4b Explanation: *<one or two sentences>*
<empty line>

[6 marks]

c. Assume now that we declare `printTranscript()` as `synchronized`, and do not change anything else. Is the above code prone to deadlocks? Explain why or why not.

Report your answer in the following format.

Q4c Answer: *<yes or no>*
Q4c Explanation: *<one or two sentences>*
<empty line>

[4 marks]

d. Assume again that we declare `printTranscript()` as `synchronized`, and do not change anything else in the above code. Is the `Student` class thread safe? Motivate your answer.

Report your answer in the following format.

Q4d Answer: *<yes or no>*
Q4d Explanation: *<one or two sentences>*
<empty line>

[4 marks]

e. Assume that we declare the `registerStudent` method as `synchronized`, and do not change anything else in the above code. Is the code prone to deadlocks after this change? Explain why or why not.

Report your answer in the following format.

Q4e Answer: *<yes or no>*
Q4e Explanation: *<one or two sentences>*
<empty line>

[3 marks]

[Total for Question 4: 25 marks]

Question 5. Consider the following code snippet.

```
public class WorkPlace {
  private int size = 100;
  private Item[] buffer = new Item[size];
  private int count = 0;
  private int in = 0;
  private int out = 0;

  public synchronized void put(Item o)
                                throws InterruptedException {
    while (count==size)
      wait();
    boolean wasEmpty = count==0;
    buffer[in] = o;
    ++count;
    in=(in+1)%size;
    if(wasEmpty) {
      notifyAll();
    }
  }

  public synchronized Item get()
                                throws InterruptedException {
    while (count==0)
      wait();
    boolean wasFull = count==size;
    Item o = buffer[out];
    buffer[out] = null;
    --count;
    out=(out+1)%size;
    if (wasFull) {
      notifyAll();
    }
    return o;
  }

  public synchronized void waitEvent()
                                throws InterruptedException {
    wait();
  }

  public synchronized void print() {
    System.out.println(Arrays.toString(buffer));
  }
}
```

```
public class Worker extends Thread {
  private WorkPlace place;

  public Worker(WorkPlace wp) {
    place = wp;
  }

  public void run(){
    Item o = produceItem();
    try {
      Thread.sleep(10000);
      while (!isItemFinished(o)) {
        if (o != null){
          place.put(o);
        }
        o = polishItem(place.get());
      }
    }
    catch (InterruptedException e){
      return;
    }
    showItem(o);
  }

  private Item produceItem(){
    // omitted code, with no call to any method in
    // the Workplace class
  }

  private boolean isItemFinished(Item o){
    // omitted code, with no call to any method in
    // the Workplace class
  }

  private Item polishItem(Item o){
    // omitted code, with no call to any method in
    // the Workplace class
  }

  private void showItem(Item o){
    System.out.println("Finished item: " + o);
  }
}
```

```
public class Checker extends Thread {
  private WorkPlace place;
  private volatile boolean stopped;

  public Checker(WorkPlace wp) {
    place = wp;
    stopped = false;
  }

  public void run() {
    while(!stopped){
      try{
        place.waitEvent();
      }
      catch(InterruptedException e){
        break;
      }
      place.print();
    }
  }

  public void stopCheck(){
    stopped = true;
  }
}
```

Consider an application that creates a single `WorkPlace` instance W, and then starts a large number of Worker threads and one Checker thread. All the threads are created passing a reference to the `WorkPlace` instance W to their respective constructors.

a. Describe the liveness properties of the application including the above code. Namely, specify whether the above code is prone to deadlocks, starvation, livelock or missed signals, using the following format.

   Q5a Answer
   Liveness problem: *<deadlock, starvation, livelock or missed signals>*
   Is the above code prone to this liveness problem? *<yes or no>*
   Explanation: *<one or two sentences>*
   <empty line>

[8 marks]

b. Would the application's behaviour change if we replace `place.waitEvent()` with `wait()` inside the `run()` method of the `Checker` class?
Report your answer in the following format.

Q5b Answer: *<yes or no>*
Q5b Explanation: *<one or two sentences>*
<empty line>

[4 marks]

c. A friend of yours notices that `WorkPlace` instances send out notifications only when one `Item` is added to a previously empty buffer, or one `Item` is removed from a previously full buffer. They then conjecture that the `buffer` array inside the `WorkPlace` instance is neither empty nor full when the `Checker` thread calls `place.print()`. Is this conjecture correct?
Report your answer in the following format.

Q5c Answer: *<yes or no>*
Q5c Explanation: *<one or two sentences>*
<empty line>

[6 marks]

d. Suppose that the `produceItem()` method is guaranteed to never return `null`.
Can any one `Worker` thread enter the waiting set of a `WorkPlace` condition queue by calling `place.put()`, and enter the same waiting set once again by calling `place.get()`? Report your answer in the following format.

Q5d Answer: *<yes or no>*
Q5d Explanation: *<one or two sentences>*
<empty line>

[4 marks]

e. Consider a multi-threaded application that does not start any `Checker` thread, but includes only a main thread and an indefinite number of `Worker` threads. A friend of yours notices that in this case, threads waiting on any one `WorkPlace`'s condition queue are all of the same type, so they argue that waking up any one waiting thread is safe in this case.
Is it indeed safe to replace `notifyAll()` with `notify()` in `WorkPlace` if we start only `Worker` threads? Report your answer in the following format.

Q5e Answer: *<yes or no>*
Q5e Explanation: *<one or two sentences>*
<empty line>

[3 marks]

[Total for Question 5: 25 marks]